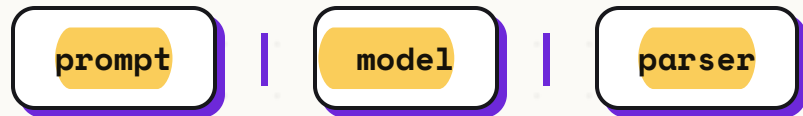


LangChain

The open-source framework for building LLM apps by **composing** small, swappable components — **prompts, models, parsers**, tools, retrievers, memory.



One API call isn't an **app**

A raw LLM call gives you text

- You still write prompt templating, output parsing, retries.
- Grounding on your data (RAG) is a pipeline, not a call.
- Tools, memory, and multiple models = lots of glue code.

LangChain gives you the glue

- Standard *components* for each piece.
- A common way to *compose* them (LCEL).
- Swap models/vector stores without rewriting your app.

In one line: LangChain is the toolkit + the wiring standard for going from a single prompt to a real, testable LLM application.

What **is** LangChain?

A framework whose building blocks are **Runnables** — anything you can `.invoke()` — that snap together into pipelines.

01
Models

Chat models & embeddings (OpenAI, etc.) behind one interface.

02
Prompts

Templates with variables and roles.

03
Output parsers

Turn model text into clean strings or typed objects.

04
Tools

Python functions the model can call.

05
Retrievers

Fetch relevant context for RAG.

06
Memory & agents

State + the loop that ties it together.

Compose with the pipe

LCEL — the LangChain Expression Language — wires components with `|`. Data flows left → right, each step a Runnable.

- Get **streaming**, **batching** & **async** for free.
- Swap any piece without touching the rest.
- The same shape scales from a one-liner to a big DAG.

01_lcel_chain.py

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

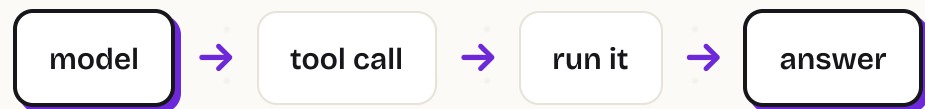
prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer in ONE sentence."),
    ("human", "Explain {topic} to a {audience}.")])

# THE CHAIN
chain = prompt | ChatOpenAI(model="gpt-4o") | StrOutputParser()
chain.invoke({"topic": "embeddings", "audience": "SRE"})
```

`prompt | model | parser` — that's LangChain in one line.

Let the model **call functions**

A tool is a plain function with `@tool`. `create_agent` hands the model the tools and runs the loop — it picks a tool, you run it, it answers.



```
02_tools_agent.py

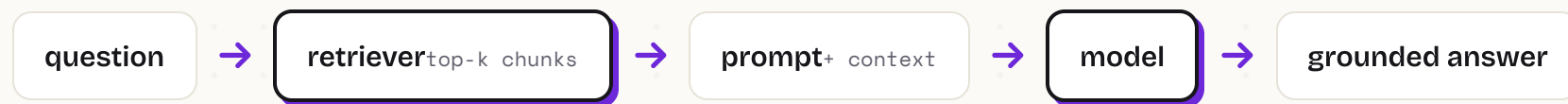
from langchain_core.tools import tool
from langchain.agents import create_agent

@tool
def get_pod_status(namespace: str) -> str:
    "Status of pods in a namespace."
    return "api-7f9: CrashLoopBackOff"

agent = create_agent("openai:gpt-4o", [get_pod_status])
agent.invoke({"messages": [user_q]})
```

Also: `.with_structured_output(Model)` → typed extraction.

RAG is just another chain



Same **composition idea**: the retriever fetches relevant chunks from a vector store, the prompt stuffs them in as context, and the model answers from *your* data — with citations. Chunk → embed → retrieve → generate.

Loaders + splitters

Docs → clean chunks.

Vector store

Embeddings + similarity search.

Retriever

A Runnable you pipe like any other.

Where teams ship it **today**

RAG

Docs / knowledge chatbots

Answer from runbooks, wikis, policies — grounded & cited.

EXTRACT

Structured extraction

Emails/tickets/PDFs → typed JSON for downstream systems.

AGENT

Tool-using assistants

Query APIs, databases, the cloud (our AWS agents).

PIPE

Summarize & classify

Batch pipelines: route tickets, tag content, digest logs.

Sweet spot: tasks that are a *straight line* of steps — input → transform → LLM → structured output.

Straight lines have **limits**

A chain is a DAG — one direction

- It can't easily *loop* (call tools, reflect, retry).
- Branching on results gets messy.
- Shared state across steps is implicit.
- Pausing for a human? Persisting progress? Hard.

Real agents need cycles



...then *loop back* to think again — until done.

Enter LangGraph → when your app needs loops, branches, shared state, memory, or a human in the middle, you graduate from a chain to a *graph*.

LangChain **vs** LangGraph

	LANGCHAIN	LANGGRAPH
Model	pipeline / DAG (a b c)	graph / state machine (nodes + edges)
Flow	linear, one direction	loops, branches, conditionals
State	passed along (implicit)	explicit, typed, shared
Memory	add-on	built-in checkpointer
Human-in-loop	manual	native interrupt()
Best for	prompts · RAG · extraction	agents · retries · approvals

They compose: LangGraph nodes run LangChain components. Not either/or — LangChain is the parts, LangGraph is the orchestration.

Reach for LangChain **when...**

✓ Use LangChain

- The flow is a straight line of steps.
- RAG, extraction, summarization, classification.
- Simple single-shot tool use.

→ Reach for LangGraph

- You need a loop (agent that retries / reflects).
- Branching, shared state, or memory.
- A human must approve mid-flow.

Rule of thumb: straight line → chain. The moment you need a *loop*, a *branch*, *memory*, or a *human* → graph.

— LIVE DEMO

Run it now

terminal

```
cd langchain-langgraph-demo
pip install -r requirements.txt
cp env.sample .env          # paste OPENAI_API_KEY

python langchain/01_lcel_chain.py      # the pipe
python langchain/02_tools_agent.py     # tools + agent
python langchain/03_structured_output.py # typed extraction
```

Then: the LangGraph session picks up exactly where the chain's limits end — turning that straight line into a looping, stateful agent.

LangChain, in a breath

- Build LLM apps by **composing components** — models, prompts, parsers, tools, retrievers.
- Wire them with **LCEL**: `prompt` | `model` | `parser`.
- Great for **RAG, extraction, summarization, simple agents**.
- Straight-line flows shine; loops & state → that's **LangGraph** (next session).

One line to remember: LangChain is the *parts and the pipe* — the standard way to assemble an LLM application.